অধ্যায় ১৪

১৪

Advanced Functions, Closures, Higher-Order Patterns এবং Functional Programming

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Function-কে মান হিসেবে ব্যবহার করতে: রাখা, পাঠানো, ফেরত দেওয়া।
- Closure দিয়ে সুরক্ষিত অবস্থা তৈরি করতে, এবং Loop-এর Closure ফাঁদ চিনতে।
- নিজের Higher-Order Function লিখতে: অন্য Function-কে মুড়ে নতুন আচরণ যোগ করা।
- `call`, `apply` ও `bind` দিয়ে `this` এবং Argument নিয়ন্ত্রণ করতে।
- ছোট Function জুড়ে Pipeline তৈরি করতে (`pipe`, `compose`)।
- Currying ও Partial Application দিয়ে পুনর্ব্যবহারযোগ্য Function তৈরি করতে।
- Pure Function ও Immutability-র সুবিধা ব্যাখ্যা করতে, এবং Side Effect সীমানায় রাখতে।
- Memoization, Debounce ও Throttle কখন উপকারী আর কখন ক্ষতিকর, তা বিচার করতে।
- একটি সম্পূর্ণ **Sales Function Lab** তৈরি ও পরীক্ষা করতে, যার Timer-নির্ভর অংশও নিয়ন্ত্রিতভাবে Test করা।

পূর্বশর্ত

- অধ্যায় ৫: Function, Arrow Function, Scope, Closure-এর প্রাথমিক ধারণা।
- অধ্যায় ৬: `map`, `filter`, `reduce`।
- অধ্যায় ১০: Timer এবং Debounce (অনুশীলনী ১২)।
- অধ্যায় ১২–১৩: ES Module এবং `this`।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১৩-এ তথ্য ও আচরণ একসঙ্গে রাখার একটি উপায় দেখেছেন: Class। এই অধ্যায় দেখায় আরেকটি, সমান শক্তিশালী উপায়: ছোট, নির্ভরযোগ্য Function তৈরি করে সেগুলো জুড়ে বড় কাজ গঠন। এই ধারার নাম **Functional Programming**। JavaScript দুটি ধারাই সমর্থন করে, এবং আধুনিক Code-এ দুটিই পাশাপাশি দেখা যায়: React-এর Component ও Hook, Redux-এর Reducer, Data Processing-এর Pipeline—সবই Functional ধারণার ওপর।

এর মূল সুবিধা নির্ভরযোগ্যতা। যে Function একই ইনপুটে সব সময় একই ফল দেয় এবং বাইরের কিছু বদলায় না, তাকে বোঝা, পরীক্ষা করা এবং পুনর্ব্যবহার করা সহজ। আর Closure ও Higher-Order Function-এর মতো কৌশল দিয়ে এমন সাধারণ সরঞ্জাম লেখা যায়—যেমন `once`, `memoize`, `throttle`—যা যেকোনো Function-এর ওপর প্রয়োগ করা যায়।

১৪.১ Function একটি মান

JavaScript-এ Function একটি **First-Class Value**: সংখ্যা বা লেখার মতোই Variable-এ রাখা যায়, Object-এর ভেতরে রাখা যায়, Argument হিসেবে পাঠানো যায়, এবং অন্য Function থেকে ফেরত দেওয়া যায়।

values.js · JavaScript — চলবে: Browser Console ও Node.js

```
function greet(name) {
  return `স্বাগতম, ${name}`;
}

const actions = { greet, shout: (text) => text.toUpperCase() };
console.log(actions.greet('Sara'), actions.shout('ok'));
console.log(greet.name, greet.length, typeof greet);

function multiplier(factor) {
  return (value) => value * factor;
}

const double = multiplier(2);
const triple = multiplier(3);
console.log(double(21), triple(7));
```

Output

```
স্বাগতম, Sara OK
greet 1 function
42 21
```

`multiplier` একটি **Function Factory**: প্রতিবার ডাকলে একটি নতুন Function তৈরি করে ফেরত দেয়, যা তৈরির সময়ের `factor` মনে রাখে। Function-এর নিজেরও Property আছে: `name` তার নাম, `length` কয়টি Parameter আশা করে (Default ও Rest Parameter বাদে)।

১৪.২ Closure: মনে রাখা অবস্থা

অধ্যায় ৫-এ দেখেছেন, একটি Function তার তৈরির জায়গার Variable মনে রাখে, এমনকি বাইরের Function শেষ হয়ে যাওয়ার পরও। এটিই **Closure**। এর সবচেয়ে কাজের ব্যবহার হলো এমন অবস্থা তৈরি, যা বাইরে থেকে সরাসরি ছোঁয়া যায় না:

closure-state.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createCounter(start = 0) {
  let count = start;

  return {
    next: () => ++count,
    reset: () => {
      count = start;
    },
    get value() {
      return count;
    },
  };
}

const tickets = createCounter(100);
const orders = createCounter();

tickets.next();
tickets.next();
orders.next();
console.log(tickets.value, orders.value);

tickets.reset();
console.log(tickets.value, 'count' in tickets);
```

Output

```
102 1
100 false
```

প্রতিটি `createCounter()` ডাকে একটি নতুন `count` তৈরি হয়, তাই `tickets` ও `orders` পরস্পরকে প্রভাবিত করে না। `count` Object-এর Property নয় (`'count' in tickets` মিথ্যা); এটিকে বদলানোর একমাত্র পথ ফেরত দেওয়া তিনটি Function। অধ্যায় ১৩-এর Private Field (`#`) যে সুরক্ষা দেয়, Closure তা দেয় Class ছাড়াই।

Loop-এর Closure ফাঁদ

loop-trap.js · JavaScript চলবে: Browser Console ও Node.js

```
const withVar = [];
for (var i = 0; i < 3; i++) {
  withVar.push(() => i);
}

const withLet = [];
for (let j = 0; j < 3; j++) {
  withLet.push(() => j);
}

console.log(withVar.map((fn) => fn()).join(', '));
console.log(withLet.map((fn) => fn()).join(', '));
```

Output

```
3, 3, 3
0, 1, 2
```

`var`-এর একটিই Variable পুরো Loop জুড়ে; তিনটি Function একই `i` মনে রাখে, এবং ডাকার সময় তার মান Loop শেষের `3`। `let` প্রতিটি পুনরাবৃত্তিতে একটি নতুন Variable তৈরি করে, তাই প্রতিটি Function নিজের মান পায়। পুরোনো Code-এ Timer বা Event Listener-এ এই Bug প্রায়ই দেখা যায়; সমাধান সহজ: `var`-এর বদলে `let`।

> **নোট:** Closure যা মনে রাখে, তা Memory-তে থেকে যায় যতক্ষণ Function-টি বেঁচে থাকে। একটি বড় তালিকা মনে রাখা Event Listener সরানো না হলে সেই তালিকাও কখনো মুক্ত হয় না। দীর্ঘজীবী Closure-এ শুধু প্রয়োজনীয় তথ্যটুকু রাখুন।

১৪.৩ Higher-Order Function: Function-কে মোড়ানো

যে Function অন্য Function নেয় বা ফেরত দেয়, তা **Higher-Order Function**। `map` ও `filter` আপনি ব্যবহার করেছেন; এবার নিজে লিখুন। সবচেয়ে কাজের ধরন হলো একটি Function-কে "মোড়ানো": মূল কাজ অপরিবর্তিত রেখে তার চারপাশে নতুন আচরণ যোগ।

wrappers.js · JavaScript চলবে: Browser Console ও Node.js

```
function once(fn) {
  let done = false;
  let result;

  return (...args) => {
    if (!done) {
      done = true;
      result = fn(...args);
```

```
    }
    return result;
  };
}

function logged(fn) {
  return (...args) => {
    const result = fn(...args);
    console.log(`${fn.name}(${args.join(', ')}) = ${result}`);
    return result;
  };
}

const connect = once(() => {
  console.log('সংযোগ তৈরি');
  return 'connection-1';
});
console.log(connect(), connect());

const add = (a, b) => a + b;
const loggedAdd = logged(add);
console.log(loggedAdd(2, 3) * 10);
```

Output

```
সংযোগ তৈরি
connection-1 connection-1
add(2, 3) = 5
50
```

- `once` নিশ্চিত করে কাজটি একবারই হবে—Database সংযোগ, Payment-এর অনুরোধ, স্বাগত-বার্তা। দ্বিতীয় ডাকে আগের ফল ফেরত আসে, কাজ আবার হয় না। `done` ও `result` Closure-এ।
- `logged` যেকোনো Function-কে না বদলে তার প্রতিটি ডাক লিখে রাখে। মোড়ানো Function আসলটির মতোই আচরণ করে (ফল ফেরত দেয়), তাই যেকোনো জায়গায় বসানো যায়।
- `...args` (Rest) সব Argument সংগ্রহ করে, `fn(...args)` (Spread) সেগুলো হুবহু পাঠায়; ফলে Wrapper জানে না, জানার দরকারও নেই, মূল Function কয়টি Argument নেয়।

১৪.৪ call, apply এবং bind

অধ্যায় ১৩-এ `bind` দিয়ে `this` স্থির করেছেন। প্রতিটি Function-এর তিনটি Method আছে, যা দিয়ে `this` নিজে বেছে দেওয়া যায়:

call-apply-bind.js · JavaScript চলবে: Browser Console ও Node.js

```
function describe(prefix, suffix) {
  return `${prefix} ${this.name}${suffix}`;
}

const user = { name: 'Sara' };

console.log(describe.call(user, 'নাম:', '।'));
console.log(describe.apply(user, ['নাম:', '!']));

const hello = describe.bind(user, 'হ্যালো,');
console.log(hello('!'));

console.log(Math.max.apply(null, [4, 9, 2]), Math.max(...[4, 9, 2]));
```

Output

```
নাম: Sara।
নাম: Sara!
হ্যালো, Sara!
9 9
```

Method	কী করে	Argument
`fn.call(thisValue, a, b)`	এখনই ডাকে	একটি একটি করে
`fn.apply(thisValue, [a, b])`	এখনই ডাকে	একটি Array-তে
`fn.bind(thisValue, a)`	ডাকে না; নতুন Function ফেরত দেয়	আগে থেকে কিছু Argument বসানো যায়

আধুনিক Code-এ `apply`-এর বেশির ভাগ কাজ Spread (`...`) করে, যেমন শেষ লাইন। কিন্তু `bind`-এর একটি দ্বিতীয় ক্ষমতা লক্ষ করুন: `hello`-তে প্রথম Argument `'হ্যালো,'` আগে থেকেই বসানো। একে বলে **Partial Application**, যা অনুচ্ছেদ ১৪.৬-এ আরও দেখবেন।

১৪.৫ Function Composition: pipe এবং compose

ছোট, একটি কাজের Function জুড়ে বড় কাজ গঠনকে বলে **Composition**। একটির ফল পরেরটির ইনপুট—কারখানার Assembly Line-এর মতো।

pipe.js · JavaScript চলবে: Browser Console ও Node.js

```
const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);

const trim = (text) => text.trim();
const lower = (text) => text.toLowerCase();
const words = (text) => text.split(/\s+/);
const count = (list) => list.length;

const wordCount = pipe(trim, lower, words, count);
const slug = pipe(trim, lower, words, (list) => list.join('-'));

console.log(wordCount('  JavaScript শিখুন প্রতিদিন  '));
console.log(slug(' Modern JavaScript Guide '));
```

Output

```
3
modern-javascript-guide
```

`pipe(f, g, h)(x)` মানে `h(g(f(x)))`, কিন্তু পড়া যায় বাঁ থেকে ডানে, কাজের ক্রমে। `pipe`-এর ভেতরটা একটি `reduce`: শুরুর মান `input`, এবং প্রতিটি Function আগের ফলকে নতুন ফলে রূপান্তর করে। লক্ষ করুন, `trim`, `lower`, `words` দুটি Pipeline-এ পুনর্ব্যবহৃত; প্রতিটি আলাদাভাবে পরীক্ষাযোগ্য।

`compose` একই কাজ করে উল্টো ক্রমে, গণিতের `f(g(x))`-এর মতো: `compose(h, g, f)` = `pipe(f, g, h)`। দুটির কোনটি ব্যবহার করবেন তা রুচির বিষয়; একটি Project-এ একটিই বেছে নিন।

Pipeline-এর শর্ত: প্রতিটি ধাপ একটি ইনপুট নেয় এবং একটি ফল দেয়। দুটি Argument লাগে এমন Function (যেমন `join(list, '-')`) সরাসরি বসানো যায় না; তখন পরের অনুচ্ছেদের কৌশল লাগে।

১৪.৬ Currying এবং Partial Application

Partial Application মানে একটি Function-এর কিছু Argument আগে বসিয়ে কম Argument-এর নতুন Function তৈরি। **Currying** মানে বহু-Argument-এর Function-কে এমনভাবে রূপান্তর, যাতে Argument এক এক করে দেওয়া যায়, এবং যতক্ষণ সব না আসে ততক্ষণ একটি নতুন Function ফেরত আসে।

curry.js · JavaScript চলবে: Browser Console ও Node.js

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...more) => curried(...args, ...more);
  };
```

```
}

const priceWithVat = curry((ratePercent, price) =>
  Math.round((price * (100 + ratePercent)) / 100));

const bdVat = priceWithVat(15);
console.log(bdVat(1000), bdVat(200));
console.log(priceWithVat(5, 1000));

const isAbove = curry((limit, value) => value > limit);
console.log([120, 2500, 950, 4000].filter(isAbove(1000)).join(', '));

const discount = (percent, price) => price * (1 - percent / 100);
const tenPercentOff = discount.bind(null, 10);
console.log(tenPercentOff(500));
```

Output

```
1150 230
1050
2500, 4000
450
```

- `bdVat` একটি Partial Application: হার আগে বসানো, দাম পরে। একটি সাধারণ Function থেকে নির্দিষ্ট কাজের Function তৈরি হলো।
- `isAbove(1000)` সরাসরি `filter`-এ বসেছে। `filter` প্রতিটি মানের জন্য একটি Argument দেয়; Currying দুই-Argument-এর তুলনাকে এক-Argument-এর প্রশ্নে পরিণত করেছে। এটিই Pipeline-এ Currying-এর আসল উপযোগিতা।
- `curry` `fn.length` দেখে বোঝে কতগুলো Argument দরকার। তাই Default বা Rest Parameter-এর Function-এ এটি নির্ভরযোগ্য নয় (তাদের `length` কম দেখায়)।
- `bind(null, 10)` একই Partial Application, Built-in উপায়ে, যখন `this` লাগে না।
- টাকার হিসাবে দশমিকের সমস্যা এড়াতে হার শতাংশে (`15`) রেখে পূর্ণসংখ্যায় হিসাব করা হয়েছে (অধ্যায় ২)।

১৪.৭ Pure Function, Immutability এবং Side Effect

একটি **Pure Function**-এর দুটি শর্ত: একই ইনপুটে সব সময় একই ফল, এবং বাইরের কিছু বদলায় না (কোনো **Side Effect** নেই)।

purity.js · JavaScript চলবে: Browser Console ও Node.js

```
let taxPercent = 15;
const impureTotal = (price) => (price * (100 + taxPercent)) / 100;
const pureTotal = (price, percent) => (price * (100 + percent)) / 100;

console.log(impureTotal(100));
taxPercent = 20;
console.log(impureTotal(100), pureTotal(100, 15));

const addTag = (product, tag) => ({ ...product, tags: [...product.tags, tag] });

const pen = Object.freeze({ name: 'Pen', tags: Object.freeze(['stationery']) });
const tagged = addTag(pen, 'sale');
console.log(pen.tags.join(','), '|', tagged.tags.join(','));
```

Output

```
115
120 115
stationery | stationery,sale
```

- **`impureTotal`** একই ইনপুট `100` -এ দুবার দুই রকম ফল দিয়েছে, কারণ এটি বাইরের `taxPercent` পড়ে। Test-এ এর ফল নির্ভর করে Test-এর আগে কোন Code চলেছে তার ওপর। **`pureTotal`** যা লাগে সব Argument হিসেবে নেয়।
- **`addTag`** মূল Product বদলায় না, নতুন একটি ফেরত দেয় (অধ্যায় ৭-এর Spread)। `Object.freeze` এর প্রমাণ: মূল Object বদলানোর চেষ্টা হলে সেটি নীরবে ব্যর্থ হতো, আর Strict Mode-এ Error দিত।

Side Effect	উদাহরণ
বাইরের Variable বদলানো	`total += x` (Function-এর বাইরের `total`)
Argument বদলানো	`items.push(x)`, `user.name = ...`
বাইরের জগতের সঙ্গে যোগাযোগ	`console.log`, `fetch`, `localStorage`, DOM
অনির্দিষ্ট ইনপুট	`Date.now()`, `Math.random()`

Immutability-র একটি পারফরম্যান্স ফাঁদ জানা দরকার। `rows.reduce((acc, row) => ({ ...acc, [row.id]: row }), {})` দেখতে সুন্দর, কিন্তু প্রতিটি ধাপে আগের সব Key-সহ একটি নতুন Object তৈরি হয়: ১০০০ সারিতে প্রায় পাঁচ লক্ষ Key Copy। ১০ সারিতে পার্থক্য চোখে পড়ে না, ১০,০০০ সারিতে Program লক্ষণীয়ভাবে ধীর। সমাধান: Function-এর ভেতরে একটি নতুন, স্থানীয় Object তৈরি করে তাতে সরাসরি লিখুন, অথবা `Object.fromEntries(rows.map((row) => [row.id, row]))`। বাইরে থেকে Function তখনও Pure, কারণ স্থানীয় Object বাইরে থেকে কেউ দেখে না যতক্ষণ না সেটি ফেরত যায়।

Side Effect ছাড়া কোনো Program কাজের নয়: শেষ পর্যন্ত কিছু দেখাতে বা সংরক্ষণ করতেই হয়। লক্ষ্য Side Effect দূর করা নয়, **সীমানায় রাখা**: হিসাবের কেন্দ্র Pure, আর Console, Network, Storage প্রান্তে, অল্প কয়েকটি জায়গায় (অধ্যায় ১২-এর Domain ও Adapter)। একটি Function-এর ভেতরে স্থানীয় Variable বদলানো Side Effect নয়, যতক্ষণ তা বাইরে থেকে দেখা যায় না।

১৪.৮ Memoization

Pure Function একই ইনপুটে একই ফল দেয়, তাই একবার হিসাব করা ফল মনে রেখে পরে সরাসরি ফেরত দেওয়া যায়। একে বলে **Memoization**। ফল মনে রাখতে এখানে `Map` ব্যবহার করা হয়েছে: Object-এর মতোই Key-Value জোড়া রাখে, তবে যেকোনো ধরনের মান Key হতে পারে, এবং `has`, `get`, `set` Method আছে।

memoize.js · JavaScript — চলবে: Browser Console ও Node.js

```
function memoize(fn) {
  const cache = new Map();

  return (n) => {
    if (!cache.has(n)) {
      cache.set(n, fn(n));
    }
    return cache.get(n);
  };
}

let calls = 0;
const slowSquare = (n) => {
  calls += 1;
  return n * n;
};

const fastSquare = memoize(slowSquare);
console.log(fastSquare(9), fastSquare(9), fastSquare(4), calls);

const fib = memoize((n) => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
console.log(fib(70));
```

Output

```
81 81 16 2
190392490709135
```

`fastSquare(9)` দ্বিতীয়বার হিসাব ছাড়াই এসেছে (`calls` মাত্র ২)। দ্বিতীয় উদাহরণ নাটকীয়: Memoization ছাড়া `fib(70)` সরল Recursive পদ্ধতিতে কয়েক হাজার কোটি Function Call করত; এখানে প্রতিটি `n` একবারই হিসাব হয়েছে।

Memoization বিনামূল্যে নয়। সতর্কতা:

- **শুধু Pure Function।** `Date.now()` বা Network-নির্ভর ফল মনে রাখলে বাসি ফল আসবে।
- **Memory বাড়ে।** প্রতিটি নতুন ইনপুট Cache-এ জমে; সীমাহীন ইনপুটে সীমাহীন Memory। প্রয়োজনে Cache-এর আকার সীমিত করুন বা মুছে ফেলার ব্যবস্থা রাখুন।
- **Key সাবধানে।** Object-কে Key বানালে `Map` তার পরিচয় (Reference) দেখে, বিষয়বস্তু নয়; একই বিষয়ের দুটি আলাদা Object দুটি আলাদা Key।
- **সস্তা হিসাবে লাভ নেই।** Cache-এ খোঁজাও সময় নেয়; শুধু সত্যিকারের ব্যয়বহুল কাজে Memoize করুন, এবং আগে মেপে দেখুন।

১৪.৯ Debounce এবং Throttle

অধ্যায় ১০-এর অনুশীলনীতে **Debounce** দেখেছেন: দ্রুত পরপর ঘটনায় শুধু শেষটির পর একবার কাজ। এর সঙ্গী **Throttle**: নির্দিষ্ট সময়ের মধ্যে সর্বোচ্চ একবার কাজ, ঘটনা যত ঘনই হোক।

throttle.js · JavaScript — চলবে: Browser Console ও Node.js

```
function throttle(fn, ms) {
  let last = -Infinity;

  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}

let runs = 0;
const onScroll = throttle(() => {
  runs += 1;
}, 180);

for (let t = 0; t < 500; t += 50) {
  setTimeout(onScroll, t);
}
setTimeout(() => console.log(`10টি ঘটনা, কাজ হলো ${runs} বার`), 600);
```

Output

```
10টি ঘটনা, কাজ হলো 3 বার
```

	Debounce	Throttle
কখন চলে	ঘটনা থামার পর একবার	চলাকালীন নিয়মিত বিরতিতে
উদাহরণ	খোঁজার Box: লেখা থামলে Request	Scroll বা Resize: চলার সময় অবস্থান হালনাগাদ
ঝুঁকি	ঘটনা না থামলে কখনো চলে না	শেষ ঘটনাটি বাদ পড়তে পারে

দুটিই Closure-এ অবস্থা রাখে (`timer` বা `last`) এবং দুটিই Higher-Order Function: যেকোনো Function-কে মুড়ে দেয়।

Functional না Object-Oriented?

একই সমস্যা দুই ধারায় লিখে তুলনা করুন। একটি Shopping Cart-এর মোট:

two-styles.js · JavaScript — চলবে: Browser Console ও Node.js

```
class Cart {
  #items = [];

  add(name, price) {
    this.#items.push({ name, price });
    return this;
  }

  get total() {
    return this.#items.reduce((sum, item) => sum + item.price, 0);
  }
}

const addItem = (items, name, price) => [...items, { name, price }];
const totalOf = (items) => items.reduce((sum, item) => sum + item.price, 0);

const cart = new Cart().add('Pen', 50).add('Notebook', 120);
const before = addItem([], 'Pen', 50);
const after = addItem(before, 'Notebook', 120);

console.log(cart.total, totalOf(after), totalOf(before));
```

Output

```
170 170 50
```

Class-এর ধারায় একটি Object নিজের অবস্থা রাখে এবং তা বদলায়; পুরোনো অবস্থা আর পাওয়া যায় না। Functional ধারায় প্রতিটি ধাপ একটি নতুন মান, তাই আগের অবস্থা (`before`) অক্ষত, যা Undo, ইতিহাস বা তুলনার জন্য স্বাভাবিক সুবিধা, এবং Test-এ প্রতিটি ধাপ আলাদাভাবে যাচাই করা যায়। অন্যদিকে Class নিয়ম ও অবস্থাকে একটি নামযুক্ত এককে আবদ্ধ রাখে, এবং দীর্ঘজীবী, অনেক নিয়মের জিনিসের (Bank Account, Booking) জন্য প্রায়ই বেশি স্বাভাবিক।

ব্যবহারিক নিয়ম: তথ্যের রূপান্তর (Report, ছাঁকা, হিসাব) Functional ধারায়; নিজস্ব জীবনচক্র ও নিয়মের সত্তা Class-এ। অধ্যায় ১২-এর কার্ট Functional ছিল, অধ্যায় ১৩-এর Booking Class; দুটি সিদ্ধান্তই তাদের সমস্যার জন্য উপযুক্ত। Timer-নির্ভর Code পরীক্ষা করতে আসল সময়ের অপেক্ষা ধীর ও অনিশ্চিত; Project-এর Test Node.js-এর **Fake Timer** দিয়ে সময়কে হাতে এগিয়ে নেয়, তাই ফল প্রতিবার নিশ্চিত ও তাৎক্ষণিক।

১৪.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Loop-এ `var` এবং Closure	সব Function শেষ মান দেখে	`let`
`fn` পাঠানোর বদলে `fn()`	ফল পাঠানো হয়, Function নয়	Parenthesis ছাড়া
Wrapper ফল ফেরত দেয় না	মোড়ানো Function `undefined` দেয়	`return fn(...args)`
Wrapper Argument বাদ দেয়	কিছু Argument হারায়	`...args`
Impure Function Memoize	বাসি বা ভুল ফল	শুধু Pure Function
সীমাহীন Cache	Memory ধীরে ধীরে বাড়ে	সীমা বা মুছে ফেলা
Default Parameter-এর Function Curry	আগেই চলে যায়	Argument-এর সংখ্যা নিজে দিন
Argument বদলানো "Pure" Function	অন্য জায়গায় অপ্রত্যাশিত পরিবর্তন	নতুন মান ফেরত দিন
অতিরিক্ত দীর্ঘ Pipeline	ত্রুটির ধাপ খুঁজে পাওয়া কঠিন	নামযুক্ত মধ্যবর্তী ধাপ
সব কিছু Functional করার জেদ	অস্পষ্ট Code	সরল `for` Loop-ও চলে, যদি পরিষ্কার হয়

১৪.১১ Debugging নির্দেশিকা

- **Pipeline-এর মাঝে দেখুন।** একটি ছোট `tap` Function বসান: `const tap = (label) => (value) => { console.log(label, value); return value; };` তারপর `pipe(trim, tap('trim-এর পর'), lower, ...)`। এটি মান না বদলে শুধু দেখায়।
- **Closure-এর অবস্থা দেখতে** DevTools-এ Function-এর ভেতরে Breakpoint দিন; Scope Panel-এ "Closure" অংশে মনে রাখা Variable দেখা যায়।
- **Stack-এ Anonymous Function** পড়া কঠিন; Wrapper-এর ভেতরের Function-কে নাম দিন, বা মূল Function-এর নাম (`fn.name`) Log করুন।
- **Memoized Function ভুল ফল দিচ্ছে?** Cache মুছে আবার চালান; ঠিক হলে সমস্যা Cache-এর Key বা Impure Function-এ।

- **Currying-এ "is not a function"** মানে সব Argument আসার আগেই আসল Function চলে গেছে, বা উল্টো; `fn.length` দেখুন।
- **Timer-নির্ভর Test মাঝে মাঝে ব্যর্থ হয়?** আসল সময়ের ওপর নির্ভর না করে Fake Timer ব্যবহার করুন।

১৪.১২ পেশাদার অভ্যাস

- **ছোট, এক কাজের, নামযুক্ত Function;** নাম পড়েই Pipeline বোঝা যায়।
- **Pure কেন্দ্র, Side Effect প্রান্তে।**
- **ইনপুট কখনো বদলাবেন না;** নতুন মান ফেরত দিন, প্রয়োজনে `Object.freeze`।
- **Wrapper সব সময় `...args` নেবে এবং ফল ফেরত দেবে।**
- **Memoize শুধু মাপা ব্যয়বহুল Pure কাজে।**
- **Debounce ব্যবহারকারীর লেখায়, Throttle ঘন ঘটনায়।**
- **Functional ও Object-Oriented মেশাতে দ্বিধা করবেন না;** যেটি সমস্যাকে স্পষ্ট করে সেটিই।
- **পরিচিত Library-র নাম ও আচরণ মানুন** (`pipe`, `compose`, `debounce`), যাতে অন্য প্রোগ্রামাররা সহজে বোঝেন।

১৪.১৩ Project: Sales Function Lab

একটি দোকানের বিক্রির তালিকা থেকে Report তৈরি: অবৈধ সারি বাদ দিয়ে কারণসহ তালিকা, মোট আয়, বিভাগভিত্তিক আয়, এবং শীর্ষ বিক্রি। পুরো হিসাব Pure Function-এর একটি Pipeline; Side Effect শুধু Report ছাপার File-এ। সঙ্গে একটি ছোট Functional সরঞ্জামের Module, যার Timer-নির্ভর অংশ Fake Timer দিয়ে পরীক্ষিত।

Project-এর চাহিদা

- প্রতিটি বিক্রি: `{ id, category, quantity, unitPrice }`; `quantity` ধনাত্মক পূর্ণসংখ্যা, `unitPrice` অঋণাত্মক সংখ্যা, `id` ও `category` খালি নয়।
- অবৈধ সারি Report থামাবে না; বাদ পড়া সারির নম্বর ও কারণ জানাবে।
- মোট আয়; বিভাগভিত্তিক সংখ্যা ও আয়, বেশি আয় আগে; শীর্ষ ৩ বিক্রি।
- ইনপুট তালিকা কখনো বদলাবে না; তৈরি হওয়া প্রতিটি সারি অপরিবর্তনীয়।
- টাকা দেখানোর Formatter একবারই তৈরি হবে, পরে পুনর্ব্যবহৃত (Memoization)।

File-এর কাঠামো

Folder-এর কাঠামো

```
sales-function-lab/
├── src/
│   ├── fp.mjs          pipe, compose, curry, memoize, once, debounce, throttle
│   ├── sales.mjs       বিক্রির নিয়ম ও Report-এর Pipeline
│   └── format.mjs      টাকা দেখানো (Memoized Formatter)
├── data/
│   └── sales.mjs       নমুনা তথ্য
├── report.mjs          Report ছাপা (Side Effect এখানে)
├── README.md
└── tests/
    ├── fp.test.mjs
    └── sales.test.mjs
```

src/fp.mjs

দুই অংশ মিলে সম্পূর্ণ File।

fp.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```js
export const pipe = (...fns) => (input) =>
  fns.reduce((value, fn) => fn(value), input);

export const compose = (...fns) => pipe(...fns.toReversed());

export function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...more) => curried(...args, ...more);
  };
}

export function memoize(fn, keyOf = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  const memoized = (...args) => {
    const key = keyOf(...args);
    if (!cache.has(key)) {
      cache.set(key, fn(...args));
    }
    return cache.get(key);
  };
  memoized.cacheSize = () => cache.size;
  return memoized;
}
```

```
// ...
```

fp.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

export function once(fn) {
  let done = false;
  let result;
  return (...args) => {
    if (!done) {
      done = true;
      result = fn(...args);
    }
    return result;
  };
}

export function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

export function throttle(fn, ms) {
  let last = -Infinity;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}
```

এই Module-এর প্রতিটি Function অধ্যায়ের কোনো না কোনো অনুচ্ছেদ থেকে এসেছে, একটি সাধারণ রূপে। `memoize` এখন যেকোনো সংখ্যক Argument নেয়, এবং Key তৈরির নিয়ম (`keyOf`) বাইরে থেকে বদলানো যায়; Default নিয়ম সব Argument-কে JSON লেখায় রূপান্তর করে। `cacheSize` Function-এর একটি Property: Function-ও Object, তাই তাতে Property যোগ করা যায়, এবং Test এটি দিয়ে যাচাই করে Cache কাজ করছে কি না।

src/sales.mjs

দুই অংশ মিলে সম্পূর্ণ File।

sales.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
import { curry, pipe } from './fp.mjs';

export class SaleError extends Error {
  constructor(message) {
    super(message);
    this.name = 'SaleError';
  }
}

// একটি সারি যাচাই: ঠিক থাকলে অপরিবর্তনীয় নতুন Object, নইলে SaleError।
export function validateSale(row) {
  const { id, category, quantity, unitPrice } = row ?? {};

  if (typeof id !== 'string' || id.trim() === '') {
    throw new SaleError('id নেই');
  }
  if (typeof category !== 'string' || category.trim() === '') {
    throw new SaleError('বিভাগ নেই');
  }
  if (!Number.isInteger(quantity) || quantity < 1) {
    throw new SaleError('পরিমাণ 1 বা বেশি পূর্ণসংখ্যা হতে হবে');
  }
  if (!Number.isFinite(unitPrice) || unitPrice < 0) {
    throw new SaleError('দাম অঋণাত্মক সংখ্যা হতে হবে');
  }
  return Object.freeze({
    id: id.trim(), category: category.trim(), quantity, unitPrice,
  });
}

export const withTotal = (sale) =>
  Object.freeze({ ...sale, total: sale.quantity * sale.unitPrice });

export const inCategory = curry((category, sale) => sale.category === category);
export const atLeast = curry((minimum, sale) => sale.total >= minimum);
export const sumBy = curry((field, rows) =>
  rows.reduce((sum, row) => sum + row[field], 0));

// ...
```

sales.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

// বৈধ সারি রাখে; অবৈধগুলোর কারণ আলাদা তালিকায়। ইনপুট বদলায় না।
export function partitionValid(rows) {
  const valid = [];
```

```
  const rejected = [];

  for (const [index, row] of rows.entries()) {
    try {
      valid.push(withTotal(validateSale(row)));
    } catch (error) {
      if (!(error instanceof SaleError)) {
        throw error;
      }
      rejected.push(`সারি ${index + 1}: ${error.message}`);
    }
  }
  return { valid, rejected };
}

export function totalsByCategory(sales) {
  return Object.entries(Object.groupBy(sales, (sale) => sale.category))
    .map(([category, rows]) => ({
      category, count: rows.length, revenue: sumBy('total', rows),
    }))
    .toSorted((a, b) =>
      b.revenue - a.revenue || a.category.localeCompare(b.category));
}

const byTotalDesc = (a, b) => b.total - a.total || a.id.localeCompare(b.id);

export const topSales = curry((count, sales) =>
  sales.toSorted(byTotalDesc).slice(0, count));

export const salesReport = pipe(
  partitionValid,
  ({ valid, rejected }) => ({
    revenue: sumBy('total', valid),
    byCategory: totalsByCategory(valid),
    top: topSales(3, valid).map((sale) => sale.id),
    rejected,
  }),
);
```

- `validateSale` **সীমানার পাহারাদার** (Trust Boundary): বাইরের সারি যাচাই করে একটি নতুন, `Object.freeze` করা Object দেয়। এরপরের প্রতিটি Function নিশ্চিন্তে ধরে নিতে পারে তথ্য বৈধ।
- `withTotal` নতুন Object তৈরি করে `total` যোগ করে; মূলটি অপরিবর্তিত।
- **Curried সাহায্যকারী:** `inCategory('Books')`, `atLeast(1000)`, `sumBy('total')` —প্রতিটি `filter`, `reduce` বা Pipeline-এ সরাসরি বসানো যায়।
- `partitionValid` **-এর ভেতরে** `push` Side Effect নয়: `valid` ও `rejected` এই Function-এর নিজের নতুন Array, বাইরে থেকে দেখা যায় না। একটি অবৈধ সারি পুরো Report থামায় না, কিন্তু অপ্রত্যাশিত ত্রুটি (Bug) চাপা পড়ে

না (অধ্যায় ১১)।

- `toSorted` (অধ্যায় ১২) ইনপুট না বদলে সাজায়; সমান আয়ে দ্বিতীয় স্তরে নাম ধরে সাজানো, যাতে ফল সব সময় নিশ্চিত ক্রমে আসে।
- `salesReport` **একটি Pipeline:** প্রথম ধাপ সারি পরিষ্কার করে, দ্বিতীয় ধাপ Report গঠন করে। নতুন একটি হিসাব যোগ করতে শুধু দ্বিতীয় ধাপে একটি লাইন।

src/format.mjs, নমুনা তথ্য এবং report.mjs

format.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { memoize } from './fp.mjs';

// Intl.NumberFormat তৈরি তুলনামূলক ব্যয়বহুল; প্রতিটি দশমিক-সংখ্যার জন্য একবারই।
export const formatterFor = memoize((decimals) =>
  new Intl.NumberFormat('en-US', {
    minimumFractionDigits: decimals,
    maximumFractionDigits: decimals,
  }));

export const formatTaka = (amount, decimals = 0) =>
  `${formatterFor(decimals).format(amount)} টাকা`;
```

sales.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export const SALES = [
  { id: 'S1', category: 'Books', quantity: 3, unitPrice: 450 },
  { id: 'S2', category: 'Electronics', quantity: 1, unitPrice: 2800 },
  { id: 'S3', category: 'Books', quantity: 1, unitPrice: 620 },
  { id: 'S4', category: 'Stationery', quantity: 10, unitPrice: 25 },
  { id: 'S5', category: 'Electronics', quantity: 2, unitPrice: 950 },
  { id: 'S6', category: 'Books', quantity: 0, unitPrice: 300 },
  { id: 'S7', category: ' ', quantity: 1, unitPrice: 100 },
  { id: 'S8', category: 'Stationery', quantity: 4, unitPrice: 120 },
];
```

report.mjs · JavaScript — চলবে: Node.js

```
import { SALES } from './data/sales.mjs';
import {
  atLeast, inCategory, partitionValid, salesReport,
} from './src/sales.mjs';
import { formatTaka, formatterFor } from './src/format.mjs';

const report = salesReport(SALES);

console.log(`মোট আয়: ${formatTaka(report.revenue)}`);
for (const { category, count, revenue } of report.byCategory) {
```

```
  console.log(`${category}: ${count}টি বিক্রি, ${formatTaka(revenue)}`);
}
console.log(`শীর্ষ ৩: ${report.top.join(', ')}`);
for (const reason of report.rejected) {
  console.log(`বাদ: ${reason}`);
}

const { valid } = partitionValid(SALES);
const bigBooks = valid.filter(inCategory('Books')).filter(atLeast(1000));
console.log(`১০০০+ টাকার বই: ${bigBooks.map((sale) => sale.id).join(', ')}`);
console.log(formatTaka(1234.5, 2), formatterFor.cacheSize(), SALES[5].quantity);
```

Output

```
মোট আয়: 7,400 টাকা
Electronics: 2টি বিক্রি, 4,700 টাকা
Books: 2টি বিক্রি, 1,970 টাকা
Stationery: 2টি বিক্রি, 730 টাকা
শীর্ষ ৩: S2, S5, S1
বাদ: সারি 6: পরিমাণ 1 বা বেশি পূর্ণসংখ্যা হতে হবে
বাদ: সারি 7: বিভাগ নেই
১০০০+ টাকার বই: S1
1,234.50 টাকা 2 0
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Side Effect একটি File-এ।** `report.mjs`-ই একমাত্র জায়গা যেখানে `console.log`; `src/`-এর সব Function Pure (Formatter-এর Cache ছাড়া, যা বাইরে থেকে অদৃশ্য এবং ফলে প্রভাব ফেলে না)। তাই Report-এর প্রতিটি সংখ্যা Test-এ সরাসরি যাচাই করা যায়।
- **ছয়টি বৈধ বিক্রি থেকে ৭,৪০০ টাকা;** দুটি সারি কারণসহ বাদ। সারির নম্বর মানুষের গণনায় (১ থেকে), Index-এ নয়।
- **বিভাগের তিনটিরই ২টি করে বিক্রি,** তাই আয়ই ক্রম ঠিক করেছে।
- **Curried Predicate-এর পুনর্ব্যবহার:** `inCategory('Books')` ও `atLeast(1000)` একই `filter` শৃঙ্খলে, পড়তে প্রায় সাধারণ বাক্যের মতো।
- **শেষ লাইন তিনটি প্রমাণ দেয়:** দশমিকসহ টাকা দেখানো; Formatter Cache-এ মাত্র দুটি (০ ও ২ দশমিক), যদিও `formatTaka` পাঁচবার ডাকা হয়েছে; এবং মূল তথ্যের অবৈধ সারির পরিমাণ এখনো `0`, অর্থাৎ ইনপুট কেউ বদলায়নি।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node report.mjs
node --test
```

Call	যা হওয়ার কথা
`validateSale({ id: 'X', category: 'A', quantity: 1.5, unitPrice: 10 })`	`SaleError` : পরিমাণের বার্তা
`salesReport([])`	আয় 0; ফাঁকা বিভাগ, শীর্ষ ও বাদের তালিকা
`topSales(2)(sales)` এবং `topSales(2, sales)`	একই ফল (Currying)
`compose(f, g)(x)`	`pipe(g, f)(x)` -এর সমান
`debounce(fn, 300)` চারবার দ্রুত	৩০০ মিলিসেকেন্ড পরে একবার, শেষ Argument দিয়ে
`throttle(fn, 100)` প্রতি ১০ মিলিসেকেন্ডে ৫০ বার	প্রতি ১০০ মিলিসেকেন্ডে একবার

Companion Source Code-এ ২২টি Test আছে। Debounce ও Throttle-এর Test Node.js-এর `mock.timers` দিয়ে সময়কে হাতে এগিয়ে নেয়, তাই এক সেকেন্ডের পরিস্থিতিও তাৎক্ষণিক ও নিশ্চিতভাবে পরীক্ষিত। Purity-র Test ইনপুট `Object.freeze` করে দেখে কোনো Function তা বদলানোর চেষ্টা করে কি না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একটি অবৈধ সারিতে পুরো Report থেমে যায়	যাচাইয়ের ত্রুটি ধরা হয়নি	`partitionValid`
Report চালানোর পর মূল তথ্য বদলে গেছে	কোথাও `sort` বা সরাসরি Assignment	`toSorted` , নতুন Object
সমান আয়ের বিভাগের ক্রম অনিশ্চিত	দ্বিতীয় স্তরের সাজানো নেই	`localeCompare`
`sumBy(...) is not a function`	সব Argument একবারে দেওয়ায় ফল পাওয়া গেছে, তারপর আবার ডাকা	Curried Function-এর ব্যবহার
Formatter-এর Cache বড় হয়ে যায়	Key-তে অপ্রয়োজনীয় মান	`keyOf`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Code-এর ভেতরে, File বা Network থেকে আসে না; টাকা সাধারণ সংখ্যায়, পয়সার হিসাব নেই; Memoized Formatter-এর Cache কখনো খালি হয় না, যা এখানে সমস্যা নয় কারণ দশমিক-সংখ্যা মাত্র কয়েকটি।

১৪.১৪ অনুশীলনী

প্রাথমিক স্তর

১. `makeGreeting(greeting)` লিখুন, যা একটি Function ফেরত দেয়; সেটি নাম নিয়ে পূর্ণ শুভেচ্ছা দেয়।

২. `var` দিয়ে লেখা Loop-এ তৈরি Function-গুলোর Output না চালিয়ে বলুন, তারপর `let` দিয়ে ঠিক করুন।

৩. `twice(fn)` লিখুন, যা একটি নতুন Function দেয়, যেটি `fn`-কে পরপর দুবার প্রয়োগ করে: `twice(double)(3)` হবে `12`।

৪. নিচের কোনটি Pure? `(a) => a * 2`, `(a) => a * Math.random()`, `(list) => list.push(1)`। কারণ বলুন।

৫. `bind` দিয়ে `calcTax(percent, amount)` থেকে `vat15` তৈরি করুন।

মধ্যম স্তর

৬. `pipe` দিয়ে নামের তালিকা পরিষ্কার করুন: আগে-পরের ফাঁকা জায়গা বাদ, প্রথম অক্ষর বড়, তারপর কমা দিয়ে জোড়া।

৭. `between(min, max, value)` Curry করে `filter`-এ ব্যবহার করুন।

৮. `memoize` দিয়ে একটি ধীর Function-এর কতবার আসল হিসাব হলো তা গুনে দেখান।

৯. `once` দিয়ে "স্বাগত Email" পাঠানোর Function লিখুন, যা তিনবার ডাকলেও একবারই পাঠায়।

চ্যালেঞ্জ স্তর

১০. `pipe` ব্যবহার করে `compose` লিখুন, এবং একই তিনটি Function-এ দুটির ক্রমের পার্থক্য দেখান।

১১. Closure দিয়ে একটি Rate Limiter লিখুন: একটি সময়-সীমার মধ্যে সর্বোচ্চ `n` বার অনুমতি দেয়। সময় একটি `clock` Function থেকে নিন, যাতে আসল সময় ছাড়াই পরীক্ষা করা যায়।

১২. Project-এ বিভাগপ্রতি গড় বিক্রি যোগ করুন: একটি Curried `averageBy(field)`, এবং প্রতিটি বিভাগের গড় `total`।

১৪.১৫ অনুশীলনীর সমাধান

সমাধান ১

greeting.js · JavaScript — চলবে: Browser Console ও Node.js

```
const makeGreeting = (greeting) => (name) => `${greeting}, ${name}!`;

const welcome = makeGreeting('স্বাগতম');
const morning = makeGreeting('শুভ সকাল');

console.log(welcome('Sara'));
console.log(morning('Rafi'));
```

Output

```
স্বাগতম, Sara!
শুভ সকাল, Rafi!
```

সমাধান ২ — `var`-এর Output `3, 3, 3`: সব Function একই `i` দেখে, যার মান Loop শেষে `3`। `let` দিলে প্রতিটি পুনরাবৃত্তির নিজস্ব Variable, ফল `0, 1, 2` (অনুচ্ছেদ ১৪.২-এর উদাহরণ)।

সমাধান ৩

twice.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const twice = (fn) => (value) => fn(fn(value));

const double = (n) => n * 2;
const exclaim = (text) => `${text}!`;

console.log(twice(double)(3));
console.log(twice(exclaim)('বাহ'));
```

Output

```
12
বাহ!!
```

সমাধান ৪ — শুধু `(a) => a * 2` Pure। `Math.random()` থাকায় দ্বিতীয়টি একই ইনপুটে ভিন্ন ফল দেয়; তৃতীয়টি তার Argument বদলায় (Side Effect), এবং ফেরত দেয় নতুন দৈর্ঘ্য, তালিকা নয়। Pure রূপ: `(list) => [...list, 1]`।

সমাধান ৫

vat-bind.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const calcTax = (percent, amount) => (amount * percent) / 100;
const vat15 = calcTax.bind(null, 15);

console.log(vat15(1000), vat15(240));
```

Output

```
150 36
```

সমাধান ৬

clean-names.js · JavaScript — চলবে: Browser Console ও Node.js

```
const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);

const trimAll = (names) => names.map((name) => name.trim());
const capitalize = (names) =>
  names.map((name) => name[0].toUpperCase() + name.slice(1));
const joinWithComma = (names) => names.join(', ');

const cleanNames = pipe(trimAll, capitalize, joinWithComma);
console.log(cleanNames(['  sara ', 'rafi', ' mina']));
```

Output

```
Sara, Rafi, Mina
```

সমাধান ৭

between.js · JavaScript — চলবে: Browser Console ও Node.js

```
const curry = (fn) => function curried(...args) {
  if (args.length >= fn.length) {
    return fn(...args);
  }
  return (...more) => curried(...args, ...more);
};

const between = curry((min, max, value) => value >= min && value <= max);
const isPassMark = between(33, 100);

console.log([12, 45, 88, 101, 33].filter(isPassMark).join(', '));
console.log(between(1)(10)(5), between(1, 10)(50));
```

Output

```
45, 88, 33
true false
```

Argument একসঙ্গে বা এক এক করে—দুভাবেই চলে।

সমাধান ৮

count-calls.js · JavaScript — চলবে: Browser Console ও Node.js

```
function memoize(fn) {
  const cache = new Map();
  return (key) => {
    if (!cache.has(key)) {
      cache.set(key, fn(key));
    }
    return cache.get(key);
  };
}

let realCalls = 0;
const shippingCost = memoize((city) => {
  realCalls += 1;
  return city === 'Dhaka' ? 60 : 120;
});

const orders = ['Dhaka', 'Bogura', 'Dhaka', 'Dhaka', 'Bogura'];
console.log(orders.map(shippingCost).join(', '));
console.log(`আসল হিসাব: ${realCalls} বার`);
```

Output

```
60, 120, 60, 60, 120
আসল হিসাব: 2 বার
```

পাঁচটি Order, কিন্তু দুটি শহর; তাই আসল হিসাব দুবার।

সমাধান ৯

welcome-once.js · JavaScript — চলবে: Browser Console ও Node.js

```
function once(fn) {
  let done = false;
  let result;
  return (...args) => {
    if (!done) {
      done = true;
      result = fn(...args);
    }
    return result;
  };
}

const sendWelcome = once((email) => {
```

```
  console.log(`স্বাগত Email পাঠানো হলো: ${email}`);
  return true;
});

sendWelcome('sara@example.com');
sendWelcome('sara@example.com');
console.log(sendWelcome('other@example.com'));
```

Output

```
স্বাগত Email পাঠানো হলো: sara@example.com
true
```

তৃতীয় ডাকে নতুন ঠিকানা দিলেও কিছু পাঠানো হয়নি; প্রথম ফল ফেরত এসেছে। বাস্তবে "প্রতি ব্যবহারকারী একবার" চাইলে ঠিকানা ধরে Memoize করাই উপযুক্ত।

সমাধান ১০

compose.js · JavaScript — চলবে: Browser Console ও Node.js

```
const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);
const compose = (...fns) => pipe(...fns.toReversed());

const add10 = (n) => n + 10;
const double = (n) => n * 2;
const square = (n) => n * n;

console.log(pipe(add10, double, square)(1));
console.log(compose(add10, double, square)(1));
```

Output

```
484
12
```

`pipe`: (১ + ১০) × ২ = ২২, তারপর বর্গ ৪৮৪। `compose` ডান থেকে: ১-এর বর্গ ১, × ২ = ২, + ১০ = ১২।

সমাধান ১১

rate-limiter.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createRateLimiter(limit, windowMs, clock = Date.now) {
  let times = [];

  return () => {
    const now = clock();
```

```
    times = times.filter((time) => now - time < windowMs);
    if (times.length >= limit) {
      return false;
    }
    times = [...times, now];
    return true;
  };
}

let fakeNow = 0;
const allow = createRateLimiter(2, 1000, () => fakeNow);

const results = [allow(), allow(), allow()];
fakeNow = 1000;
results.push(allow());
console.log(results.join(', '));
```

Output

```
true, true, false, true
```

প্রথম সেকেন্ডে দুবার অনুমতি, তৃতীয়টি প্রত্যাখ্যাত; ১০০০ মিলিসেকেন্ড পরে পুরোনো দুটি সময়-সীমার বাইরে চলে যায়, তাই আবার অনুমতি। `clock` Inject করায় এক সেকেন্ড অপেক্ষা না করেই পরীক্ষা হয়েছে, Project-এর Fake Timer-এর মতোই।

সমাধান ১২

average.mjs · JavaScript — চলবে: Node.js

```
import { curry } from '../../project/sales-function-lab/src/fp.mjs';
import {
  partitionValid, sumBy,
} from '../../project/sales-function-lab/src/sales.mjs';
import { SALES } from '../../project/sales-function-lab/data/sales.mjs';

const averageBy = curry((field, rows) =>
  (rows.length === 0 ? 0 : Math.round(sumBy(field, rows) / rows.length)));

const { valid } = partitionValid(SALES);
const groups = Object.groupBy(valid, (sale) => sale.category);

for (const [category, rows] of Object.entries(groups)) {
  console.log(`${category}: গড় ${averageBy('total', rows)}`);
}
```

Output

```
Books: গড় 985
Electronics: গড় 2350
Stationery: গড় 365
```

ফাঁকা তালিকায় শূন্য দিয়ে ভাগ এড়াতে Guard রাখা হয়েছে; পুরোনো কোনো Function বদলাতে হয়নি।

১৪.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. "Function একটি First-Class Value"—এর অর্থ কোনটি?

ক) Function সবার আগে চলে

খ) Function-কে অন্য মানের মতো রাখা, পাঠানো ও ফেরত দেওয়া যায়

গ) Function শুধু Class-এর ভেতরে থাকে

ঘ) Function-এর নাম বাধ্যতামূলক

প্রশ্ন ২. `var` দিয়ে `for` Loop-এ তিনটি Function তৈরি করে পরে ডাকলে, প্রতিটি `i` হিসেবে কী দেখে?

ক) Loop শেষের মান

খ) নিজের পুনরাবৃত্তির মান

গ) `undefined`

ঘ) Error

প্রশ্ন ৩. `fn.bind(obj, 5)` কী ফেরত দেয়?

ক) `fn`-এর ফল

খ) `obj`

গ) একটি নতুন Function, যার `this` `obj` এবং প্রথম Argument `5`

ঘ) `undefined`

প্রশ্ন ৪. `pipe(f, g, h)(x)` কীসের সমান?

ক) `f(g(h(x)))`

খ) `h(g(f(x)))`

গ) `[f(x), g(x), h(x)]`

ঘ) `f(x) + g(x) + h(x)`

প্রশ্ন ৫. `add(1)(2)` এবং `add(1, 2)` দুটিই কাজ করে। `add` কোন কৌশলে তৈরি?

ক) Memoization

খ) Debounce

গ) Currying

ঘ) Closure ছাড়া সাধারণ Function

প্রশ্ন ৬. কোনটি Pure Function?

ক) `(price) => price * (1 + taxRate)`, যেখানে `taxRate` বাইরের Variable

খ) `(list) => list.sort()`

গ) `() => Date.now()`

ঘ) `(price, rate) => price * (1 + rate)`

প্রশ্ন ৭. কোন Function Memoize করা নিরাপদ নয়?

ক) Network থেকে সর্বশেষ দাম আনা

খ) একটি সংখ্যার বর্গ

গ) Fibonacci সংখ্যা

ঘ) একটি শহরের নির্দিষ্ট Delivery Fee

প্রশ্ন ৮. Scroll চলাকালীন প্রতি ২০০ মিলিসেকেন্ডে সর্বোচ্চ একবার অবস্থান হালনাগাদ করতে কোনটি?

ক) Debounce

খ) Throttle

গ) Memoize

ঘ) Curry

প্রশ্ন ৯. `once(fn)`-এর তৈরি Function দ্বিতীয়বার ডাকলে কী হয়?

ক) `fn` আবার চলে

খ) Error

গ) `undefined`

ঘ) `fn` চলে না; প্রথম ফল ফেরত আসে

প্রশ্ন ১০. Debounce-এর Test আসল সময়ের অপেক্ষা ছাড়া নিশ্চিতভাবে করার উপায়?

ক) `setTimeout`-এর সময় বাড়ানো

খ) Test একাধিকবার চালানো

গ) Fake Timer দিয়ে সময় হাতে এগিয়ে নেওয়া

ঘ) Debounce-এর Test সম্ভব নয়

১৪.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–খ · ৫–গ · ৬–ঘ · ৭–ক · ৮–খ · ৯–ঘ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Function সংখ্যা বা লেখার মতোই একটি মান।
২	ক	`var`-এর একটিই Variable; সব Function তার শেষ মান দেখে।
৩	গ	`bind` ডাকে না; `this` ও কিছু Argument স্থির করা নতুন Function দেয়।
৪	খ	`pipe` বাঁ থেকে ডানে: প্রথমে `f`, শেষে `h`।
৫	গ	Argument এক এক করে বা একসঙ্গে দেওয়া যায়।
৬	ঘ	সব ইনপুট Argument-এ, বাইরের কিছু পড়ে বা বদলায় না; `sort` তালিকা বদলায়।
৭	ক	ফল সময়ের সঙ্গে বদলায়; Cache বাসি দাম দেবে।
৮	খ	ঘটনা চলাকালীন নিয়মিত বিরতিতে কাজ: Throttle।
৯	ঘ	`done` Closure-এ মনে রাখে কাজ হয়ে গেছে।
১০	গ	Fake Timer সময়কে নিয়ন্ত্রিত ও তাৎক্ষণিক করে।

১৪.১৮ অধ্যায়ের সারসংক্ষেপ

- Function একটি মান: রাখা, পাঠানো, ফেরত দেওয়া যায়; Function Factory নির্দিষ্ট কাজের Function তৈরি করে।
- Closure অবস্থা মনে রাখে এবং বাইরে থেকে সুরক্ষিত রাখে; Loop-এ `var` ফাঁদ, `let` সমাধান।
- Higher-Order Function দিয়ে Function মোড়ানো: `once`, `logged`; Wrapper `...args` নেয় ও ফল ফেরত দেয়।
- `call` ও `apply` এখনই ডাকে, `bind` নতুন Function দেয়; `bind` দিয়ে Partial Application।
- `pipe` বাঁ থেকে ডানে, `compose` ডান থেকে বাঁয়ে ছোট Function জুড়ে বড় কাজ।
- Currying বহু-Argument-এর Function-কে Pipeline ও `filter`-এর উপযোগী করে।
- Pure Function: একই ইনপুটে একই ফল, কোনো Side Effect নয়; Side Effect প্রান্তে।
- Memoization Pure ও ব্যয়বহুল কাজে; Cache-এর Memory ও Key-র দিকে নজর।
- Debounce ঘটনা থামার পর, Throttle চলাকালীন নির্দিষ্ট বিরতিতে; Timer-নির্ভর Code Fake Timer দিয়ে Test।

পরের অধ্যায়ে Browser API: URL, Web Storage, Fetch, File ও Blob—Browser যে সুবিধাগুলো দেয়, এবং বাস্তব Web Application-এ সেগুলোর নিরাপদ ব্যবহার।

১৪.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
First-Class Function	মান হিসেবে ব্যবহারযোগ্য Function।
Function Factory	নতুন Function তৈরি করে ফেরত দেওয়া Function।
Closure	তৈরির জায়গার Variable মনে রাখা Function।
Higher-Order Function	Function নেয় বা ফেরত দেয় এমন Function।
Wrapper	মূল Function-কে মুড়ে নতুন আচরণ যোগ করা Function।
Partial Application	কিছু Argument আগে বসিয়ে নতুন Function।
Currying	Argument এক এক করে নেওয়ার রূপান্তর।
Composition	ছোট Function জুড়ে বড় কাজ (`pipe`, `compose`)।
Pure Function	একই ইনপুটে একই ফল, কোনো Side Effect নেই।
Side Effect	Function-এর বাইরের জগতে পরিবর্তন বা যোগাযোগ।
Memoization	ফল মনে রেখে পুনরায় হিসাব এড়ানো।
Throttle	নির্দিষ্ট সময়ে সর্বোচ্চ একবার চালানো।

১৪.২০ আরও পড়ুন

- **MDN — Closures:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures` — Closure-এর বিস্তারিত ও প্রচলিত ভুল।
- **MDN — Function.prototype.call():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/call` — `call`, `apply`, `bind`।
- **MDN — Map:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map` — Key-Value সংগ্রহ।
- **Wikipedia — Pure function:** `https://en.wikipedia.org/wiki/Pure_function` — Pure Function-এর সংজ্ঞা ও উদাহরণ।
- **Node.js — Mocking timers in tests:** `https://nodejs.org/api/test.html#class-mocktimers` — `mock.timers` দিয়ে Timer-নির্ভর Test।